Practice 11 — CI/CD with GitHub Actions

Objective: put last week's app in a GitHub repository and let a **pipeline** do the work: lint and test every pull request on two Python versions, block merging until the checks are green, scan for leaked secrets, then **build, scan (Trivy) and push** the image to GHCR on every push to `main` and every `v*` tag — with meaningful tags. Stretch: a **deploy** job with a manual approval, and a **rollback** by re-deploying an older tag. These are the skills from today's lecture.

Timebox: ~55–60 min, in class. If you're stuck on one step for more than 10 minutes, ask instead of pushing through alone.

> **How you're checked:** this practice is done in class and **checked orally** at the end. There's no written report. Keep the repository's **Actions**, **Pull requests**, **Packages** and **Settings** pages open and be ready to **show** each item in the Definition of done and **explain** what each part of your workflow does. The instructor will ask you a few of the questions under "Be ready to explain".

> **What you need:** a GitHub account, `git`, Python 3.12 (for running the tests locally) and Docker. Make the repository **public**: public repositories get GitHub Actions minutes and GHCR packages for free.

Definition of done (things you can show)

☐ Your repo `lab11-cicd` with the app, tests and `.github/workflows/ci.yml`
☐ A **pull request** whose checks went **red** because a test failed, the fixing commit, and the same PR **green**
☐ The checks list on a PR: `lint`, `test (3.11)`, `test (3.12)`, `secrets-scan` — and the test-report **artifacts** of a run
☐ **Branch protection** (a ruleset) on `main`: no direct pushes, the four checks required before merging
☐ A push to `main` and a tag `v1.0.0` that ran the **`image`** job: the GHCR package page with tags **`main`**, **`sha-…`** and **`1.0.0`**
☐ The **Trivy** step in a run: what it found, and how you made the gate pass (fixed, or accepted in `.trivyignore` **with a reason**)
☐ `docker run` of the pushed `1.0.0` image on your laptop answering `/version` → `1.0.0`
☐ *(Stretch)* the `deploy` workflow waiting for **your approval** in the `production` environment, a deploy of `1.0.1` and a rollback to `1.0.0`

Part 0 — The repository (~10 min)

1. On GitHub: **New repository** → name `lab11-cicd`, **Public**, no README. Clone it and add the files below.

`app.py`:

```
import os
import socket

from flask import Flask

app = Flask(__name__)
VERSION = os.environ.get("APP_VERSION", "dev")


def greet(name: str) -> str:
    """Small pure function so we have something to unit-test."""
    return f"Hello, {name.strip().title()}!"


@app.get("/")
def index():
    return {"host": socket.gethostname(), "greeting": greet("narxoz")}


@app.get("/healthz")
def healthz():
    return {"status": "ok"}


@app.get("/version")
def version():
    return {"version": VERSION}


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000)
```

`tests/test_app.py`:

```
from app import app, greet


def test_greet():
    assert greet("  aidana ") == "Hello, Aidana!"


def test_healthz():
    client = app.test_client()
    resp = client.get("/healthz")
    assert resp.status_code == 200
    assert resp.get_json() == {"status": "ok"}


def test_version_default():
    client = app.test_client()
    assert client.get("/version").get_json()["version"] == "dev"
```

`requirements.txt`:

```
flask==3.1.3
```

`requirements-dev.txt`:

```
-r requirements.txt
pytest==8.4.2
ruff==0.14.3
```

`pyproject.toml`:

```
[tool.pytest.ini_options]
pythonpath = ["."]
testpaths = ["tests"]

[tool.ruff]
line-length = 100
```

`Dockerfile` (last week's hardened image):

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
ARG APP_VERSION=dev
ENV APP_VERSION=$APP_VERSION
RUN useradd --create-home appuser
USER appuser
EXPOSE 8000
HEALTHCHECK --interval=10s --timeout=3s --retries=3 \
  CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/healthz')"
CMD ["python", "app.py"]
```

`.dockerignore`:

```
.venv/
__pycache__/
.pytest_cache/
.ruff_cache/
.git/
.github/
tests/
.env
```

`.gitignore`:

```
.venv/
__pycache__/
.pytest_cache/
.ruff_cache/
*.xml
```

`.trivyignore` (empty for now — Part C explains it):

```
# CVEs we accept on purpose, one per line, each with a reason above it.
```

2. Run the checks **locally first** — the pipeline will run exactly these:

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
ruff check . && ruff format --check .
pytest -q                     # 3 passed
```

3. Commit and push **to `main`** (the only direct push you'll do today):

```
git add -A && git commit -m "feat: lab11 app with tests"
git push -u origin main
```

Part A — CI on every pull request (~15 min)

Create `.github/workflows/ci.yml`. The first three jobs are CI; the `image` job (Part C) only runs on pushes. Fill in the three `TODO`s — all are on the lecture slides.

```yaml
name: ci

on:
  pull_request:
  push:
    branches: [main]
    tags: ["v*"]

permissions:
  contents: read            # least privilege for every job by default

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true  # a new push cancels the old run of the same branch

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: "3.12"
          cache: pip
          cache-dependency-path: requirements-dev.txt
      - run: pip install -r requirements-dev.txt
      - run: ruff check .
      - run: ruff format --check .

  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        # TODO 1: run this job on Python "3.11" and "3.12"
        python-version: []
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: ${{ matrix.python-version }}
          cache: pip
          cache-dependency-path: requirements-dev.txt
      - run: pip install -r requirements-dev.txt
      - run: pytest -q --junitxml=report-${{ matrix.python-version }}.xml
      - uses: actions/upload-artifact@v7
        if: always()        # upload the report even when tests fail
```

```
      with:
        name: test-report-${{ matrix.python-version }}
        path: report-${{ matrix.python-version }}.xml

  secrets-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
        with:
          fetch-depth: 0    # scan the whole history, not only the last commit
      - name: gitleaks
        run: |
          docker run --rm -v "$PWD:/repo" zricethezav/gitleaks:v8.30.1 \
            detect --source /repo --redact --exit-code 1

  image:
    needs: [lint, test, secrets-scan]
    if: github.event_name == 'push'
    runs-on: ubuntu-latest
    permissions:
      contents: read
      # TODO 2: this job pushes to GHCR — give GITHUB_TOKEN write access to packages
    steps:
      - uses: actions/checkout@v7
      - name: Image name must be lower case
        run: echo "IMAGE=ghcr.io/${GITHUB_REPOSITORY,,}" >> "$GITHUB_ENV"
      - uses: docker/setup-buildx-action@v4
      - uses: docker/login-action@v4
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - id: meta
        uses: docker/metadata-action@v6
        with:
          images: ${{ env.IMAGE }}
          tags: |
            type=sha
            type=semver,pattern={{version}}
            type=ref,event=branch
      - name: Build (load locally for the scan)
        uses: docker/build-push-action@v7
        with:
          context: .
          load: true
          tags: lab11-app:scan
          build-args: APP_VERSION=${{ steps.meta.outputs.version }}
```

```
          cache-from: type=gha
          cache-to: type=gha,mode=max
      - name: Trivy gate
        uses: aquasecurity/trivy-action@v0.36.0
        with:
          image-ref: lab11-app:scan
          severity: HIGH,CRITICAL
          ignore-unfixed: true
          # TODO 3: make this step FAIL the job when it finds something
          exit-code: "0"
          trivyignores: .trivyignore
      - name: Push
        uses: docker/build-push-action@v7
        with:
          context: .
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          build-args: APP_VERSION=${{ steps.meta.outputs.version }}
          cache-from: type=gha
```

Commit it on a **branch** and open a pull request:

```
git switch -c ci
git add .github && git commit -m "ci: lint, test matrix, secret scan, image"
git push -u origin ci
```

On GitHub: **Compare & pull request**. Open the **Checks** tab and watch `lint`, `test (3.11)`, `test (3.12)` and `secrets-scan`. The `image` job is **skipped** on PRs — that's the `if:`.

Red, then green

Break the code on the same branch — remove `.strip()` from `greet`:

```
    return f"Hello, {name.title()}!"
```

```
git commit -am "refactor: simplify greet"   # a "harmless" change
git push
```

`test (3.11)` and `test (3.12)` turn **red**: open the failing step and find `test_greet`. Download the **test-report** artifact from the run's Summary page. Then fix it (`git revert HEAD` or put `.strip()` back), push again, and watch the PR turn **green**. **Don't merge yet.**

Part B — Protect `main` (~5 min)

Settings → Rules → Rulesets → New branch ruleset:

- Name `protect-main`, **Enforcement: Active**, target the default branch.
- ✅ **Restrict deletions**, ✅ **Block force pushes**
- ✅ **Require a pull request before merging** (0 approvals is fine for a solo repo — in a team you'd require 1)
- ✅ **Require status checks to pass** → add `lint`, `test (3.11)`, `test (3.12)`, `secrets-scan`

(Older UI: **Settings → Branches → Add branch protection rule** — same options.) A check only appears in the list after it has run once, which is why Part A came first.

Prove it: `git switch main && echo x >> README.md && git commit -am x && git push` → **rejected**. Undo with `git reset --hard origin/main`.

Now **merge** the PR. The push to `main` starts the full workflow **including** `image`.

Part C — Build, scan, push (~15 min)

1. Open the run on `main`. The `image` job will most likely **fail at the Trivy gate** — the Debian base image has known vulnerabilities. Read the table in the log: package, CVE, installed version, **fixed version**.

2. Fix what can be fixed. In the `Dockerfile`, right after `FROM`:

   ```
   RUN apt-get update && apt-get -y upgrade && rm -rf /var/lib/apt/lists/*
   ```

3. If something is still listed whose fixed version is **not in the mirror yet**, you may accept it — **with a reason and an owner** — in `.trivyignore`:

   ```
   # zlib1g: fixed version not yet in the Debian mirror (checked 2026-11-08).
   # Owner: <your name>. Re-check weekly; remove when the base image has the fix.
   CVE-2026-76844
   CVE-2026-85091
   ```

   (These two were the leftovers when we tested on 2026-11-08. Your list will differ — new CVEs appear every day. **Never** "fix" the gate by setting `exit-code: "0"`.)

4. Commit on a branch → PR → green → merge. The `image` job on `main` now pushes. Open your profile → **Packages** → `lab11-cicd`: tags `main` and `sha-xxxxxxx`.

5. Release `v1.0.0`:

   ```
   git switch main && git pull
   git tag v1.0.0 && git push origin v1.0.0
   ```

The tag run adds the tag `1.0.0` (metadata-action strips the `v`).

6. Pull and run it like a server would:

```
docker run --rm -d --name lab11 -p 8011:8000 ghcr.io/<your-user>/lab11-cicd:1.0.0
curl -s localhost:8011/version     # {"version":"1.0.0"}
docker rm -f lab11
```

A public package needs no login to pull. If you get `denied`, open the package → **Package settings** → **Change visibility** → **Public**.

Part D — Stretch: deploy with approval, and roll back

Option 1 — a Linux VM you can SSH into (university VM, any free-tier VM, or a local Multipass/VirtualBox VM with a public address):

1. On the VM: Docker + Compose installed, and last week's `server/` folder copied to `~/lab11` with `.env`: `IMAGE=ghcr.io/<your-user>/lab11-cicd` and `TAG=1.0.0`.
2. Create a **dedicated SSH key** for deployments (never your personal one): `ssh-keygen -t ed25519 -f deploy_key -N ""`, add `deploy_key.pub` to the VM's `~/.ssh/authorized_keys`.
3. **Settings** → **Environments** → **New environment** `production` → ✅ **Required reviewers** → yourself. Add **environment secrets** `DEPLOY_HOST`, `DEPLOY_USER`, `DEPLOY_SSH_KEY` (the private key).
4. Add `.github/workflows/deploy.yml`:

```yaml
name: deploy

on:
  workflow_dispatch:
    inputs:
      tag:
        description: "Image tag to deploy (e.g. 1.0.1, or an older one to roll back)"
        required: true

permissions:
  contents: read

concurrency:
  group: deploy-production
  cancel-in-progress: false   # never cancel a deploy halfway

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production   # waits for a reviewer, unlocks the env secrets
    steps:
      - name: Deploy over SSH
        uses: appleboy/ssh-action@v1.2.5
        env:
          TAG: ${{ inputs.tag }}
        with:
          host: ${{ secrets.DEPLOY_HOST }}
          username: ${{ secrets.DEPLOY_USER }}
          key: ${{ secrets.DEPLOY_SSH_KEY }}
          envs: TAG
          script: |
            set -euo pipefail
            cd ~/lab11
            sed -i "s/^TAG=.*/TAG=${TAG}/" .env
            docker compose pull
            docker compose up -d
            docker compose ps
```

5. Change the message, PR → merge → tag `v1.0.1`. Then **Actions → deploy → Run workflow** with `1.0.1` → **Review deployments → Approve**. Check `/version` on the VM → `1.0.1`.
6. **Roll back:** run `deploy` again with `1.0.0`. The image already exists — nothing is rebuilt; the rollback takes seconds.

Option 2 — no VM: a self-hosted runner on your laptop. Settings → Actions → Runners → New self-hosted runner and follow the commands for your OS. In `deploy.yml` use `runs-on: self-hosted` and replace the SSH step with:

```
      - env:
          TAG: ${{ inputs.tag }}
        run: |
          IMAGE="ghcr.io/$(echo "$GITHUB_REPOSITORY" | tr '[:upper:]' '[:lower:]')"
          docker rm -f lab11-prod 2>/dev/null || true
          docker run -d --name lab11-prod -p 8012:8000 "$IMAGE:$TAG"
```

(`tr` instead of `${VAR,,}`: macOS ships an old bash that doesn't know `,,`.)

> ⚠️ A self-hosted runner executes the workflow's code **on your machine**. On a **public** repository anyone can open a pull request — never let PR workflows run on a self-hosted runner, and remove the runner after class (**Settings** → **Actions** → **Runners** → **Remove**, then `./config.sh remove`).

Cleanup

- Stop the self-hosted runner and remove it (Option 2).
- On the VM: `docker compose down` (keep or delete the VM as you like).
- Delete the `production` environment secrets if the VM goes away, and delete `deploy_key` / `deploy_key.pub` from your laptop.
- Keep the repository: it's a good start for your final project's pipeline.

Be ready to explain

1. What's the difference between **CI**, **continuous delivery** and **continuous deployment**? Which ones did you build today?
2. Walk me through your workflow: which **event** started which **jobs**, and why does `image` not run on pull requests?
3. What does the **matrix** do, and what does `cache: pip` save?
4. Why `permissions: contents: read` at the top, and `packages: write` only on the `image` job? What is `GITHUB_TOKEN`?
5. Which tags did `metadata-action` create for a push to `main` and for `v1.0.0`, and which one would you deploy?
6. What did the Trivy gate find, what did you fix, and why is a `.trivyignore` entry without a reason a red flag?
7. How does branch protection turn a red check into "you can't merge"?
8. *(Stretch)* What does the `production` environment add, and how did you roll back — why was it so fast?

Common mistakes

- **A check never appears in the ruleset's list.** It has to run at least once first; the name must match exactly (`test (3.11)`, with the space).
- **`denied: installation not allowed to Write organization package`** or **`permission_denied: write_package`.** The `image` job is missing `packages: write` (TODO 2), or the package already exists and isn't linked to this repo: package → **Package settings** → **Manage Actions access** → **Add repository** with **Write**.
- **`repository name must be lowercase`.** Your GitHub user or repo has capitals — that's what the "Image name must be lower case" step is for.
- **The Trivy step shows findings but the job is green.** `exit-code` is still `"0"` (TODO 3).
- **The `image` job is skipped on `main`.** One of `lint`, `test` or `secrets-scan` failed — `needs:` stops everything after a red job.
- **gitleaks fails on your own commit.** You committed a token or key. Rotate it **first**, then remove it from history; don't just delete the line.
- **`ruff format --check` fails.** Run `ruff format .` locally and commit; run the same commands locally before every push.
- **The tag run builds `1.0.0` but the deploy says `manifest unknown`.** You typed `v1.0.0` — metadata-action drops the `v`.
- **The deploy waits forever.** It's waiting for **your** approval: open the run → **Review deployments**.